

KING SAUD UNIVERSITY
COLLEGE OF COMPUTER & INFORMATION SCIENCES
DEPARTMENT OF COMPUTER SCIENCE

LEXICAL ANALYZER

Driven by Regular Expressions

CSC339 Theory of Computation
Group Term Project

Second Semester 1446/47 AH
Due: May 9, 2026

Name	ID
Sara ALShahrani	445200266
Dana AlMutairi	445200310
Aroob bin libdah	445202255

Section: 44084

Signed Pledge: We confirm that this project write-up and implementation are our own work, and any consulted sources are acknowledged properly.

Sara ALShahrani: _____

Dana AlMutairi: _____

Aroob bin libdah: _____

Table of Contents

Table of Contents.....	1
1. Introduction.....	3
2. Project Requirements and Token Set.....	3
3. Token Specifications and Priority.....	4
4. Regular Expression Handling	4
4.1 Supported Regular Expression Features	4
4.2 Regex to Postfix Conversion	4
5. NFA Construction.....	5
6. Combining NFAs.....	6
7. NFA to DFA Conversion.....	6
7.1 Epsilon Closure.....	6
7.2 Move Function.....	6
7.3 Subset Construction	6
7.4 Accepting DFA States and Priority Handling.....	7
8. Lexer Simulation and Tokenization Strategy.....	7
8.1 Lexer Simulation Pseudocode.....	7
8.2 Main Program Flow	7
9. Data Structures Used	8
9.1 Regex and NFA Data Structures.....	8
9.2 DFA Construction Data Structures	8
9.3 Tokenization and Program Execution Data Structures	8
10. Time and Space Complexity.....	9
11. Sample Runs	10
11.1 Sample Run 1: Manual Input	10
11.2 Sample Run 2: File Input	10
12. Limitations and Future Improvements.....	11
12.1 Limitations	11
12.2 Future Improvements.....	12
13. Conclusion	12
Appendix A: Source Code Listing.....	13
TokenSpec.java.....	13
Token.java.....	13

State.java.....	14
NFA.java.....	14
RegexParser.java.....	14
NFABuilder.java.....	17
DFASState.java.....	18
NFAtoDFA.java.....	19
Lexer.java	20
Main.java	22
Appendix B: User Manual	24
1. How to Run the Program	24
2. Input Format	24
3. Output Format.....	24
4. Example Output	24
5. Error Message	24
Appendix C: Work Distribution.....	25

1. Introduction

The objective of this project is to implement a lexical analyzer driven by regular expressions. The program reads a list of token specifications, where each token has a name and a regular expression. Each regular expression is converted into an NFA, the token NFAs are combined, and the combined NFA is converted into a DFA. Finally, the DFA is simulated on an input string to produce a stream of tokens.

The implementation follows the required pipeline:

Token Specifications → Regex to NFA → Combined NFA → DFA → Lexer Simulation → Token Stream

2. Project Requirements and Token Set

The lexical analyzer recognizes the six required token classes: spaces, keywords, identifiers, numbers, operators, and delimiters. Spaces are treated as separators during tokenization.

Category	Token(s) / Regular Expression
Spaces	Used as separators
Keywords	if, then, else, while, return, for, break, continue, int, float
Identifier	ID = [A-Za-z][A-Za-z0-9]*
Number	NUM = [0-9]+(\.[0-9]+)?
Operators	+, -, *, /, =, ==, !=, <=, >=, <, >
Delimiters	(,), {, }, ;, ,,

Alphabet / Character Set Assumption: The analyzer assumes ASCII-based input including uppercase and lowercase letters, digits, whitespace, punctuation symbols, operators, delimiters, dot, and underscore.

3. Token Specifications and Priority

The token specifications are stored in TokenSpec.java as an ordered list of token names and regular expressions. The order of this list represents token priority during tie-breaking.

Priority rule: the lexer first applies longest match. If two or more tokens match the same longest lexeme, the token that appears earlier in TokenSpec is selected.

Example	Explanation
Input: if	Matches both KW_IF and ID. KW_IF is selected because keywords appear before identifiers.
Input: ==	EQ is selected instead of two ASSIGN tokens because EQ matches the longer prefix.

4. Regular Expression Handling

4.1 Supported Regular Expression Features

- Concatenation, such as ab
- Union, such as a|b
- Kleene star, such as a*
- Plus, such as a+
- Optional, such as a?
- Parentheses for grouping, such as (ab|c)*
- Character classes, such as [A-Z], [a-z], and [0-9]
- Escape handling, such as \+, *, \[, and \)

4.2 Regex to Postfix Conversion

To simplify processing, each regular expression is first converted from infix notation to postfix notation. This removes the need for parentheses during evaluation and preserves operator precedence.

```
Algorithm RegexToPostfix(regex)
Input: regular expression regex
Output: postfix expression
Begin
    regex ← preprocess(regex)
    regex ← addConcatOperators(regex)
    create empty stack S
    create empty string output
    for each character c in regex do
        if c is an operand then
            append c to output
        else if c = '(' then
            push c into S
        else if c = ')' then
            while top(S) ≠ '(' do
                append pop(S) to output
            end while
            pop '(' from S
        else if c is an operator then
            while S is not empty AND top(S) ≠ '(' AND precedence(top(S)) ≥ precedence(c) do
                append pop(S) to output
            end while
            push c into S
    end for
    while S is not empty do
        append pop(S) to output
    end while
    return output
```

```

        end while
        push c into S
    end if
end for
while S is not empty do
    append pop(S) to output
end while
return output
End

```

5. NFA Construction

After converting a regular expression to postfix, an NFA is built using Thompson's construction. A stack is used to store partial NFAs while scanning the postfix expression.

```

Algorithm BuildNFA(postfix, token, priority)
Input: postfix expression, token name, token priority
Output: NFA for the token
Begin
    create empty stack S
    for each symbol c in postfix do
        if c is an operand then
            create start state s
            create end state e
            add transition from s to e using c
            push NFA(s, e) into S
        else if c = '.' then
            NFA2 ← pop(S)
            NFA1 ← pop(S)
            connect NFA1.end to NFA2.start using  $\epsilon$ -transition
            push combined NFA into S
        else if c = '|' then
            NFA2 ← pop(S)
            NFA1 ← pop(S)
            create new start state s
            create new end state e
            connect s to NFA1.start and NFA2.start using  $\epsilon$ 
            connect NFA1.end and NFA2.end to e using  $\epsilon$ 
            push NFA(s, e) into S
        else if c = '*' then
            NFA1 ← pop(S)
            create new start state s and new end state e
            connect s to NFA1.start and e using  $\epsilon$ 
            connect NFA1.end to NFA1.start and e using  $\epsilon$ 
            push NFA(s, e) into S
        else if c = '+' then
            NFA1 ← pop(S)
            create new start state s and new end state e
            connect s to NFA1.start using  $\epsilon$ 
            connect NFA1.end to NFA1.start and e using  $\epsilon$ 
            push NFA(s, e) into S
        else if c = '?' then
            NFA1 ← pop(S)
            create new start state s and new end state e
            connect s to NFA1.start and e using  $\epsilon$ 
            connect NFA1.end to e using  $\epsilon$ 
            push NFA(s, e) into S
        end if
    end for
    result ← pop(S)
    mark result.end as accepting state
    assign token name and priority
    return result
End

```

6. Combining NFAs

Each token regular expression is first converted into a separate ϵ -NFA. After building all token NFAs, a new global start state is created. An ϵ -transition is added from this new start state to the start state of every token NFA. This creates one combined NFA that represents the full lexical specification.

```
Algorithm Combine_NFAs(NFAs)
Input: list of token NFAs
Output: global start state of the combined NFA
Begin
    create newStart state
    for each NFA in NFAs do
        add epsilon transition from newStart to NFA.start
    end for
    return newStart
End
```

7. NFA to DFA Conversion

7.1 Epsilon Closure

```
Algorithm EpsilonClosure(states)
closure  $\leftarrow$  states
stack  $\leftarrow$  states
while stack is not empty do
    s  $\leftarrow$  pop(stack)
    for each state t in s.epsilon do
        if t not in closure then
            add t to closure
            push t to stack
        end if
    end for
end while
return closure
```

7.2 Move Function

```
Algorithm Move(states, symbol)
result  $\leftarrow$  empty set
for each state s in states do
    if transition on symbol exists then
        add reachable states to result
    end if
end for
return result
```

7.3 Subset Construction

```
Algorithm ConvertToDFA(start)
startClosure  $\leftarrow$  EpsilonClosure({start})
create DFA state from startClosure
add DFA state to queue
while queue is not empty do
    current  $\leftarrow$  dequeue(queue)
    for each input symbol do
        moveSet  $\leftarrow$  Move(current.states, symbol)
        closure  $\leftarrow$  EpsilonClosure(moveSet)
        if closure is a new DFA state then
            create new DFA state
            add to queue
        end if
        add transition from current state to the new or existing DFA state
    end for
end while
return DFA start state
```

7.4 Accepting DFA States and Priority Handling

A DFA state is accepting if it contains at least one accepting NFA state. If multiple accepting token states exist inside the same DFA state, the token with the highest priority is selected. In the implementation, the smallest priority value represents the highest priority.

8. Lexer Simulation and Tokenization Strategy

The Lexer class performs the final simulation step. It receives the DFA start state and scans the input string from left to right. At each position, the lexer follows DFA transitions character by character until no more transition exists. During scanning, it stores the last accepting DFA state reached, which allows it to apply the longest match rule.

- Scan the input string
- Follow DFA transitions
- Apply longest match
- Create Token objects
- Track line and column position
- Print Lexing Completed or Lexing Error

8.1 Lexer Simulation Pseudocode

```
Algorithm LexerSimulation(input, DFAstart)
create empty token list
pos ← 0
line ← 1
column ← 1
while pos < input.length do
    if input[pos] is a space then
        skip it
        update column
        continue
    end if
    currentState ← DFAstart
    lastAcceptingState ← null
    lastAcceptingPos ← -1
    startPos ← pos
    startLine ← line
    startColumn ← column
    while transition exists do
        move to next DFA state
        advance input position
        if currentState is accepting then
            store it as lastAcceptingState
        end if
    end while
    if no accepting state was reached then
        print Lexing Error
        stop
    end if
    lexeme ← input[startPos ... lastAcceptingPos]
    tokenType ← lastAcceptingState.tokenName
    create Token(lexeme, tokenType, startLine, startColumn)
    add Token to token list
    pos ← lastAcceptingPos
    update line and column
end while
return token list
```

8.2 Main Program Flow

1. Get token specifications from TokenSpec.

2. Convert each regex to postfix using RegexParser.
3. Build an NFA for each token using NFABuilder.
4. Combine all token NFAs.
5. Convert the combined NFA into a DFA.
6. Read the input string manually or from input.txt.
7. Create a Lexer using the DFA start state.
8. Tokenize the input.
9. Print the token stream.

9. Data Structures Used

9.1 Regex and NFA Data Structures

Class / Structure	Purpose
State	Represents an NFA state with an id, character transitions, epsilon transitions, token name, and priority if accepting.
NFA	Represents an automaton with a start state and an end accepting state.
Stack	Used in regex to postfix conversion and in building the NFA from postfix.
Map and List	Used to store transitions and epsilon connections efficiently.
StringBuilder	Used to construct postfix expressions efficiently.

9.2 DFA Construction Data Structures

Class / Structure	Purpose
DFASState	Represents one DFA state as a set of NFA states with transitions, accepting status, token name, and priority handling.
Set<State>	Represents DFA states, epsilon closures, and move results.
Map<Character, DFASState>	Stores DFA transitions, where each symbol maps to the next DFA state.
Queue<DFASState>	Used for BFS traversal during subset construction.
List<DFASState>	Stores all generated DFA states to avoid duplicates.

9.3 Tokenization and Program Execution Data Structures

Class / Structure	Purpose
TokenSpec	Stores the ordered list of token specifications. Each specification contains a token name and regular expression. The list order represents token priority.
Token	Represents one token produced by the lexer, storing the lexeme, token type, line number, and column number.
List<Token>	Stores the final token stream produced by the lexer before printing.
Lexer	Uses the DFA start state to scan the input, apply longest match, track positions, and produce tokens.
Main	Acts as the driver class that connects token specifications, automata construction, lexer simulation, and output printing.

10. Time and Space Complexity

Algorithm	Time Complexity	Space Complexity
Regex to Postfix Conversion	$O(n)$	$O(n)$
NFA Construction	$O(n)$	$O(n)$
Epsilon Closure	$O(N + E)$	$O(N)$
Move Function	$O(N)$	$O(N)$
Subset Construction	$O(2^N \times A \times (N + E))$	$O(2^N)$
Lexer Simulation	$O(m)$	$O(k)$

Where n = length of the regular expression, N = number of NFA states, E = number of epsilon transitions, A = alphabet size, m = length of the input string, and k = number of tokens produced.

11. Sample Runs

11.1 Sample Run 1: Manual Input

Input method: The user selects option 1 and enters the input directly in the console.

Input: `int count = 10; while count >= 1`

Console screenshot showing the manual input and the produced token stream:

```
MINUS -> -
MULT -> \*
DIV -> /
LPAREN -> \(
RPAREN -> \)
LBRACE -> \{
RBRACE -> \}
SEMI -> ;
COMMA -> ,

===== INPUT METHOD =====
1. Enter input manually
2. Read input from input.txt
Choose option 1 or 2: 1

===== ENTER INPUT STRING =====
Input: int count = 10; while count >= 1

===== INPUT STRING =====
int count = 10; while count >= 1

===== TOKEN STREAM =====
+-----+-----+-----+
| Lexeme | Token | Position |
+-----+-----+-----+
| 'int' | KW_INT | Line 1, col 1 |
| 'count' | ID | Line 1, col 5 |
| '=' | ASSIGN | Line 1, col 11 |
| '10' | NUM | Line 1, col 13 |
| ';' | SEMI | Line 1, col 15 |
| 'while' | KW_WHILE | Line 1, col 17 |
| 'count' | ID | Line 1, col 23 |
| '>=' | GE | Line 1, col 29 |
| '1' | NUM | Line 1, col 32 |
+-----+-----+-----+

Lexing Completed
```

Figure 1. Sample Run 1 using manual console input.

11.2 Sample Run 2: File Input

Input method: The user selects option 2 and the program reads the input from input.txt.

Input file screenshot:

```
Main.java input.txt x
1int count = 10;
2while count >= 1 { count = count - 1; }
3if count == 0 then return count;
```

Figure 2. input.txt file used for Sample Run 2.

Console output screenshot showing the input read from the file and the produced token stream:

```
===== INPUT STRING =====
int count = 10;
while count >= 1 { count = count - 1; }
if count == 0 then return count;

===== TOKEN STREAM =====
```

Lexeme	Token	Position
'int'	KW_INT	Line 1, col 1
'count'	ID	Line 1, col 5
'='	ASSIGN	Line 1, col 11
'10'	NUM	Line 1, col 13
','	SEMI	Line 1, col 15
'while'	KW_WHILE	Line 2, col 1
'count'	ID	Line 2, col 7
'>='	GE	Line 2, col 13
'1'	NUM	Line 2, col 16
'{'	LBRACE	Line 2, col 18
'count'	ID	Line 2, col 20
'='	ASSIGN	Line 2, col 26
'count'	ID	Line 2, col 28
'-'	MINUS	Line 2, col 34
'1'	NUM	Line 2, col 36
','	SEMI	Line 2, col 37
'}'	RBRACE	Line 2, col 39
'if'	KW_IF	Line 3, col 1
'count'	ID	Line 3, col 4
'=='	EQ	Line 3, col 10
'0'	NUM	Line 3, col 13
'then'	KW_THEN	Line 3, col 15
'return'	KW_RETURN	Line 3, col 20
'count'	ID	Line 3, col 27
','	SEMI	Line 3, col 32

Lexing Completed

Figure 3. Sample Run 2 using file input from input.txt.

12. Limitations and Future Improvements

12.1 Limitations

- The lexical analyzer is designed for the required token categories: spaces, keywords, identifiers, numbers, operators, and delimiters.
- The current version focuses on lexical analysis only, not syntax analysis or semantic analysis.
- When an invalid character is found, the lexer reports the first offending character and its position.

- The implementation follows the required $\text{Regex} \rightarrow \text{NFA} \rightarrow \text{DFA} \rightarrow \text{Simulation}$ pipeline.

12.2 Future Improvements

- Extend the analyzer to support comments and string literals.
- Add file-based input in addition to direct user input.
- Improve error recovery so the lexer can continue scanning after reporting an invalid character.
- Add more token categories if the analyzer is expanded for a larger programming language.

13. Conclusion

In this project, we implemented a lexical analyzer driven by regular expressions. The analyzer begins with token specifications, converts each regular expression into an NFA, combines all token NFAs, converts the combined NFA into a DFA, and finally simulates the DFA on an input string.

The lexer applies the longest match rule and uses token priority to resolve conflicts. The final output is a token stream containing the lexeme, token name, and position information. This project demonstrates how regular expressions, NFAs, DFAs, and lexical analysis are connected in compiler design.

Appendix A: Source Code Listing

The full source code is also included in the submitted ZIP file. The source listing is provided below.

TokenSpec.java

```
package theory;

import java.util.*;

public class TokenSpec {

    public static List<String[]> getSpecs() {

        return List.of(

            new String[]{"KW_IF", "if"},
            new String[]{"KW_THEN", "then"},
            new String[]{"KW_ELSE", "else"},
            new String[]{"KW_WHILE", "while"},
            new String[]{"KW_RETURN", "return"},
            new String[]{"KW_FOR", "for"},
            new String[]{"KW_BREAK", "break"},
            new String[]{"KW_CONTINUE", "continue"},
            new String[]{"KW_INT", "int"},
            new String[]{"KW_FLOAT", "float"},

            new String[]{"ID", "[A-Za-z][A-Za-z0-9_]*"},
            new String[]{"NUM", "[0-9]+(\\.[0-9]+)?"},

            new String[]{"EQ", "=="},
            new String[]{"LE", "<="},
            new String[]{"GE", ">="},
            new String[]{"NE", "!="},

            new String[]{"ASSIGN", "="},
            new String[]{"LT", "<"},
            new String[]{"GT", ">"},

            new String[]{"PLUS", "\\+"},
            new String[]{"MINUS", "-"},
            new String[]{"MULT", "\\*"},
            new String[]{"DIV", "/"},

            new String[]{"LPAREN", "\\("},
            new String[]{"RPAREN", "\\)"},
            new String[]{"LBRACE", "\\{"},
            new String[]{"RBRACE", "\\}"},
            new String[]{"SEMI", ";"},
            new String[]{"COMMA", ","}

        );
    }
}
```

Token.java

```
package theory;

public class Token {

    public String lexeme;
    public String type;
    public int line;
    public int column;

    public Token(String lexeme, String type, int line, int column) {
        this.lexeme = lexeme;
        this.type = type;
        this.line = line;
        this.column = column;
    }
}
```

```

@Override
public String toString() {
    return "" + lexeme + "' ' " + type + " Line " + line + ", col " + column;
}
}

```

State.java

```

package theory;

import java.util.*;

public class State {

    public int id;
    public Map<Character, List<State>> transitions = new HashMap<>();
    public List<State> epsilon = new ArrayList<>();
    public String tokenName = null;
    public int priority = Integer.MAX_VALUE;

    public State(int id) {
        this.id = id;
    }

    public void addTransition(char c, State s) {
        transitions.computeIfAbsent(c, k -> new ArrayList<>()).add(s);
    }

    public void addEpsilon(State s) {
        epsilon.add(s);
    }
}

```

NFA.java

```

package theory;

public class NFA {

    public State start, end;

    public NFA(State s, State e) {
        start = s;
        end = e;
    }
}

```

RegexParser.java

```

package theory;

import java.util.*;

public class RegexParser {

    public static final char LIT_PLUS = '\uE000';
    public static final char LIT_STAR = '\uE001';
    public static final char LIT_LPAREN = '\uE002';
    public static final char LIT_RPAREN = '\uE003';
    public static final char LIT_DOT = '\uE004';
    public static final char LIT_LBRACE = '\uE005';
    public static final char LIT_RBRACE = '\uE006';

    static boolean isOperator(char c) {
        return c == '|' || c == '*' || c == '+' || c == '?' || c == '.';
    }

    static boolean isRegexParenthesis(char c) {
        return c == '(' || c == ')';
    }

    static boolean isOperand(char c) {

```

```

        return !isOperator(c) && !isRegexParenthesis(c);
    }

    static int precedence(char c) {
        if (c == '*' || c == '+' || c == '?') return 3;
        if (c == '.') return 2;
        if (c == '|') return 1;
        return 0;
    }

    public static char restoreLiteral(char c) {
        if (c == LIT_PLUS) return '+';
        if (c == LIT_STAR) return '*';
        if (c == LIT_LPAREN) return '(';
        if (c == LIT_RPAREN) return ')';
        if (c == LIT_DOT) return '.';
        if (c == LIT_LBRACE) return '{';
        if (c == LIT_RBRACE) return '}';
        return c;
    }

    private static char escapedToLiteral(char c) {
        if (c == '+') return LIT_PLUS;
        if (c == '*') return LIT_STAR;
        if (c == '(') return LIT_LPAREN;
        if (c == ')') return LIT_RPAREN;
        if (c == '.') return LIT_DOT;
        if (c == '{') return LIT_LBRACE;
        if (c == '}') return LIT_RBRACE;
        return c;
    }

    private static String expandCharacterClass(String content) {
        List<Character> chars = new ArrayList<>();

        for (int i = 0; i < content.length(); i++) {
            char start = content.charAt(i);

            if (i + 2 < content.length() && content.charAt(i + 1) == '-') {
                char end = content.charAt(i + 2);

                for (char c = start; c <= end; c++) {
                    chars.add(c);
                }

                i += 2;
            } else {
                chars.add(start);
            }
        }

        StringBuilder result = new StringBuilder();
        result.append('(');

        for (int i = 0; i < chars.size(); i++) {
            result.append(chars.get(i));

            if (i != chars.size() - 1) {
                result.append('|');
            }
        }

        result.append(')');
        return result.toString();
    }

    private static String preprocess(String regex) {
        StringBuilder result = new StringBuilder();

        for (int i = 0; i < regex.length(); i++) {
            char c = regex.charAt(i);

```

```

        if (c == '\\') {
            if (i + 1 < regex.length()) {
                char next = regex.charAt(i + 1);
                result.append(escapedToLiteral(next));
                i++;
            }
        }

        else if (c == '[') {
            StringBuilder content = new StringBuilder();
            i++;

            while (i < regex.length() && regex.charAt(i) != ']') {
                content.append(regex.charAt(i));
                i++;
            }

            result.append(expandCharacterClass(content.toString()));
        }

        else {
            result.append(c);
        }
    }

    return result.toString();
}

static String addConcat(String regex) {
    StringBuilder result = new StringBuilder();

    for (int i = 0; i < regex.length(); i++) {
        char c1 = regex.charAt(i);
        result.append(c1);

        if (i + 1 < regex.length()) {
            char c2 = regex.charAt(i + 1);

            boolean left = isOperand(c1) || c1 == ')' || c1 == '*' || c1 == '+' || c1 == '?';
            boolean right = isOperand(c2) || c2 == '(';

            if (left && right) {
                result.append('.');
            }
        }
    }

    return result.toString();
}

public static String toPostfix(String regex) {

    regex = preprocess(regex);
    regex = addConcat(regex);

    Stack<Character> stack = new Stack<>();
    StringBuilder output = new StringBuilder();

    for (char c : regex.toCharArray()) {

        if (isOperand(c)) {
            output.append(c);
        }

        else if (c == '(') {
            stack.push(c);
        }

        else if (c == ')') {
            while (!stack.isEmpty() && stack.peek() != '(') {
                output.append(stack.pop());
            }
        }
    }
}

```



```

        if (!stack.isEmpty()) {
            stack.pop();
        }
    }

    else if (isOperator(c)) {
        while (!stack.isEmpty()
            && stack.peek() != '('
            && precedence(stack.peek()) >= precedence(c)) {
            output.append(stack.pop());
        }

        stack.push(c);
    }
}

while (!stack.isEmpty()) {
    output.append(stack.pop());
}

return output.toString();
}
}

```

NFABuilder.java

```

package theory;

import java.util.*;

public class NFABuilder {

    static int id = 0;

    static State newState() {
        return new State(id++);
    }

    private static boolean isOperator(char c) {
        return c == '.' || c == '|' || c == '*' || c == '+' || c == '?';
    }

    private static NFA createLiteralNFA(char c) {
        State s = newState();
        State e = newState();
        s.addTransition(RegexParser.restoreLiteral(c), e);
        return new NFA(s, e);
    }

    public static NFA build(String postfix, String token, int priority) {

        Stack<NFA> stack = new Stack<>();

        for (char c : postfix.toCharArray()) {

            if (!isOperator(c)) {
                stack.push(createLiteralNFA(c));
            }

            else if (c == '.') {
                NFA b = stack.pop();
                NFA a = stack.pop();
                a.end.addEpsilon(b.start);
                stack.push(new NFA(a.start, b.end));
            }

            else if (c == '|') {
                NFA b = stack.pop();
                NFA a = stack.pop();
                State s = newState();
                State e = newState();

```

```

        s.addEpsilon(a.start);
        s.addEpsilon(b.start);
        a.end.addEpsilon(e);
        b.end.addEpsilon(e);
        stack.push(new NFA(s, e));
    }

    else if (c == '*') {
        NFA a = stack.pop();
        State s = newState();
        State e = newState();
        s.addEpsilon(a.start);
        s.addEpsilon(e);
        a.end.addEpsilon(a.start);
        a.end.addEpsilon(e);
        stack.push(new NFA(s, e));
    }

    else if (c == '+') {
        NFA a = stack.pop();
        State s = newState();
        State e = newState();
        s.addEpsilon(a.start);
        a.end.addEpsilon(a.start);
        a.end.addEpsilon(e);
        stack.push(new NFA(s, e));
    }

    else if (c == '?') {
        NFA a = stack.pop();
        State s = newState();
        State e = newState();
        s.addEpsilon(a.start);
        s.addEpsilon(e);
        a.end.addEpsilon(e);
        stack.push(new NFA(s, e));
    }
}

NFA result = stack.pop();
result.end.tokenName = token;
result.end.priority = priority;
return result;
}
}

```

DFASState.java

```

package theory;

import java.util.*;

public class DFASState {

    public Set<State> nfaStates;
    public Map<Character, DFASState> transitions = new HashMap<>();
    public boolean isAccepting = false;
    public String tokenName = null;

    public DFASState(Set<State> states) {

        this.nfaStates = states;
        int bestPriority = Integer.MAX_VALUE;

        for (State s : states) {
            if (s.tokenName != null && s.priority < bestPriority) {
                bestPriority = s.priority;
                this.tokenName = s.tokenName;
                this.isAccepting = true;
            }
        }
    }
}

```

```
}
```

NFAtoDFA.java

```
package theory;

import java.util.*;

public class NFAtoDFA {

    public static State combineNFAs(List<NFA> nfas) {
        State newStart = new State(-1);

        for (NFA nfa : nfas) {
            newStart.addEpsilon(nfa.start);
        }

        return newStart;
    }

    public static Set<State> epsilonClosure(Set<State> states) {
        Stack<State> stack = new Stack<>();
        Set<State> closure = new HashSet<>(states);

        for (State s : states) {
            stack.push(s);
        }

        while (!stack.isEmpty()) {
            State current = stack.pop();

            for (State next : current.epsilon) {
                if (!closure.contains(next)) {
                    closure.add(next);
                    stack.push(next);
                }
            }
        }

        return closure;
    }

    public static Set<State> move(Set<State> states, char c) {
        Set<State> result = new HashSet<>();

        for (State s : states) {
            if (s.transitions.containsKey(c)) {
                result.addAll(s.transitions.get(c));
            }
        }

        return result;
    }

    public static DFAState convertToDFA(State start) {
        Set<State> startSet = new HashSet<>();
        startSet.add(start);
        Set<State> startClosure = epsilonClosure(startSet);
        DFAState startDFA = new DFAState(startClosure);

        Queue<DFAState> queue = new LinkedList<>();
        List<DFAState> allStates = new ArrayList<>();

        queue.add(startDFA);
        allStates.add(startDFA);

        while (!queue.isEmpty()) {
            DFAState current = queue.poll();
```

```

        Set<Character> alphabet = new HashSet<>();

        for (State s : current.nfaStates) {
            alphabet.addAll(s.transitions.keySet());
        }

        for (char c : alphabet) {

            Set<State> moveSet = move(current.nfaStates, c);
            Set<State> closure = epsilonClosure(moveSet);

            if (closure.isEmpty()) {
                continue;
            }

            DFAState nextState = null;

            for (DFAState d : allStates) {
                if (d.nfaStates.equals(closure)) {
                    nextState = d;
                    break;
                }
            }

            if (nextState == null) {
                nextState = new DFAState(closure);
                allStates.add(nextState);
                queue.add(nextState);
            }

            current.transitions.put(c, nextState);
        }
    }

    return startDFA;
}
}

```

Lexer.java

```

package theory;

import java.util.*;

public class Lexer {

    private DFAState startState;
    private boolean hasError = false;
    private String errorMessage = "";

    public Lexer() {

    }

    public Lexer(DFAState startState) {
        this.startState = startState;
    }

    public void setStartState(DFAState startState) {
        this.startState = startState;
    }

    public List<Token> tokenize(String input) {

        List<Token> tokens = new ArrayList<>();
        hasError = false;
        errorMessage = "";

        int pos = 0;
        int line = 1;
        int column = 1;
    }
}

```

```

while (pos < input.length()) {

    char currentChar = input.charAt(pos);

    if (currentChar == ' ') {
        pos++;
        column++;
        continue;
    }

    if (currentChar == '\n') {
        pos++;
        line++;
        column = 1;
        continue;
    }

    if (currentChar == '\r' || currentChar == '\t') {
        pos++;
        column++;
        continue;
    }

    if (startState == null) {
        hasError = true;
        errorMessage = "Lexing Error: DFA start state is not initialized.";
        break;
    }

    int startPos = pos;
    int startLine = line;
    int startColumn = column;

    DFAState currentState = startState;
    DFAState lastAcceptingState = null;
    int lastAcceptingPos = -1;
    int tempPos = pos;

    while (tempPos < input.length()) {

        char c = input.charAt(tempPos);

        if (c == ' ' || c == '\n' || c == '\r' || c == '\t') {
            break;
        }

        if (!currentState.transitions.containsKey(c)) {
            break;
        }

        currentState = currentState.transitions.get(c);
        tempPos++;

        if (currentState.isAccepting) {
            lastAcceptingState = currentState;
            lastAcceptingPos = tempPos;
        }
    }

    if (lastAcceptingState == null) {

        hasError = true;
        errorMessage =
            "Lexing Error at Line "
            + line
            + ", col "
            + column
            + ": '"
            + currentChar
            + "'";

        break;
    }
}

```

```

    }

    String lexeme = input.substring(startPos, lastAcceptingPos);

    Token token = new Token(
        lexeme,
        lastAcceptingState.tokenName,
        startLine,
        startColumn
    );

    tokens.add(token);

    while (pos < lastAcceptingPos) {
        char consumed = input.charAt(pos);

        if (consumed == '\n') {
            line++;
            column = 1;
        } else {
            column++;
        }

        pos++;
    }
}

return tokens;
}

public void printTokens(List<Token> tokens) {

    System.out.println("+-----+-----+-----+");
    System.out.printf("| %-14s | %-14s | %-20s |%n", "Lexeme", "Token", "Position");
    System.out.println("+-----+-----+-----+");

    for (Token t : tokens) {
        String position = "Line " + t.line + ", col " + t.column;

        System.out.printf("| %-14s | %-14s | %-20s |%n",
            "'" + t.lexeme + "'",
            t.type,
            position);
    }

    System.out.println("+-----+-----+-----+");

    if (hasError) {
        System.out.println();
        System.out.println(errorMessage);
    } else {
        System.out.println();
        System.out.println("Lexing Completed");
    }
}
}

```

Main.java

```

package theory;

import java.util.*;
import java.nio.file.*;
import java.io.IOException;

public class Main {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
        List<String[]> specs = TokenSpec.getSpecs();
        List<NFA> nfes = new ArrayList<>();
    }
}

```

```

System.out.println("==== TOKEN SPECIFICATIONS =====");

for (int i = 0; i < specs.size(); i++) {

    String tokenName = specs.get(i)[0];
    String regex = specs.get(i)[1];
    int priority = i;

    System.out.println(tokenName + " -> " + regex);

    String postfix = RegexParser.toPostfix(regex);
    NFA nfa = NFABuilder.build(postfix, tokenName, priority);
    nfas.add(nfa);
}

System.out.println();

State combinedStart = NFAtoDFA.combineNFAs(nfas);
DFAState dfaStart = NFAtoDFA.convertToDFA(combinedStart);

String input = "";

System.out.println("==== INPUT METHOD =====");
System.out.println("1. Enter input manually");
System.out.println("2. Read input from input.txt");
System.out.print("Choose option 1 or 2: ");

String choice = scanner.nextLine().trim();

if (choice.equals("1")) {

    System.out.println();
    System.out.println("==== ENTER INPUT STRING =====");
    System.out.print("Input: ");
    input = scanner.nextLine();

} else if (choice.equals("2")) {

    String filePath = "input.txt";

    try {
        input = Files.readString(Paths.get(filePath));
    } catch (IOException e) {
        System.out.println("Error: Could not read input.txt.");
        System.out.println("Make sure input.txt is placed in the project folder.");
        scanner.close();
        return;
    }

} else {

    System.out.println("Invalid choice. Program terminated.");
    scanner.close();
    return;
}

System.out.println();
System.out.println("==== INPUT STRING =====");
System.out.println(input);
System.out.println();

Lexer lexer = new Lexer(dfaStart);
List<Token> tokens = lexer.tokenize(input);

System.out.println("==== TOKEN STREAM =====");
lexer.printTokens(tokens);
scanner.close();
}
}

```

Appendix B: User Manual

1. How to Run the Program

- Open the project in Eclipse or any Java IDE.
- Make sure all source files are inside the same package: theory.
- To use file input, create input.txt in the project folder.
- Run Main.java.

2. Input Format

The program allows the user to either enter the input manually or read the input from input.txt. Example input:

```
if x > y then {x=y; z=x-1} else x=y+5
```

3. Output Format

The program prints a token stream. Each row contains the lexeme, token name, and position information.

4. Example Output

Lexeme	Token	Position
'if'	KW_IF	Line 1, col 1
'x'	ID	Line 1, col 4
'>'	GT	Line 1, col 6

Lexing Completed

5. Error Message

If no token matches the current character, the program prints Lexing Error. The error message includes the offending character and its position.

Appendix C: Work Distribution

The following table summarizes each team member's main role and contribution in the project.

Student Name	ID	Main Responsibility	Contribution Summary
Sara ALShahrani	445200266	Regex handling and NFA construction	Implemented and documented regular expression preprocessing, postfix conversion, Thompson NFA construction, and related data structures.
Dana AlMutairi	445200310	Tokenization, lexer simulation, and program execution	Implemented and documented TokenSpec, Token, Lexer, Main, token priority, maximal munch simulation, output formatting, sample runs, and user manual.
Aroob bin libdah	445202255	NFA combination and DFA construction	Implemented and documented combining token NFAs, epsilon closure, move function, subset construction, DFAState, accepting state priority handling, and complexity analysis.